Assignment 02

C++ Object-oriented Programming

Khiem Nguyen Lecturer in Multiscale Materials khiem.nguyen@glasgow.ac.uk

1 Foreword

It is a shame that we have only one semester to learn so much new knowledge but we don't have a real opportunity to really develop a meaningful and serious project.

You might feel "What the heck! The assignment is so long." Trust me bro! *The solution for this project is much much shorter than the description itself. It is also mostly based on what you have done in Assignment 01.* The long description is to help you to understand the project better and deeper. A short description will leave you pretty confused.

1.1 Ultimate goal

In this assignment, we aim to build a mini-library that mimic two important machine learning frameworks in the machine learning world. In fact, they look ridiculously simple, they are one of the backbones of vast number of machine learning frameworks that have been used these days, including of course the genAI frameworks you are deploying. In the end of the assignment, we try to mimic two important classes provided in the celebrated library for machine learning: **scikit-learn** – Text contains hyperref. Particularly, we focus on two regression models

- Linear Regression
- Logistic Regression

1.2 Objectives

The two main objectives of this assignment consists of

1. Develop a mini-library `sklearn_lite` that mimic a small part of the Python library scikit-learn
2. Apply our built library to perform some fancy applications.

Besides these two main objectives, we take this opportunity to learn some knowledge that could be useful for you when you may choose ENG4200 and ENG5337

2 Quick tour to the world of machine learning

As mentioned above, we will develop two classes `LinearRegression` and `LogisticRegression` representing two basic regression models "Linear Regression" and "Logistic Regression", respectively.

Linear Regression. is used to make prediction on the continous spectrum of data points.

Examples

- Predict the house price of a given flat given its features such as total living area, number of bedrooms, distance to the city center, distance to schools, distance to public transporation stations, ...

- Predict the age of a person by using their recent photos.
- Predict the price of a bottle of wine given its information about the wine: number of years, location, type of grapdes, ...

Linear regression can be easily extended to **Polynomial Regression** to enhance its power and widen its capability in various complex applications.

Logistic Regression is used to classify data points into finite number of categories, such as $0, 1, 2, \dots, K$.

Examples

- Given an image of one digit written by hand, predict what number it represents. The image represent 1 out of 10 digits: $0, 1, 2, \dots, 9$.
- Given a patient's record of ECG diagram, predict whether the patient is fine or not, i.e., just 0 (not sick) or 1 (sick).

2.1 Terminologies

Before going into the mathematics behind these two machine learning models, we need some terminologies.

Supervised Learning

Both of these regression methods are classified as **supervised learning** methods in which we have the input data given by $\mathbf{x}^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)})$ and output data given by $y^{(i)}$ for $i = 1, \dots, m$, where m is the number of samples.

Dataset

Assume that we are given a dataset $\mathcal{D}$ with m **samples**, each sample has n input **features** $\mathbf{x}^{(i)} = (x_1^{(i)}, \dots, x_n^{(i)}) \in \mathbb{R}^n$ and m label $y^{(i)}$, where $i \in \{1, \dots, m\}$ is an index of one sample. So the dataset $\mathcal{D}$ can be written as follows

$$\mathcal{D} = \left\{ (\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(m)}, y^{(m)}) \right\} \tag{1}$$

Regression Problem versus Classification Problem

- In the world of machine learning & statistics, a **regression problem** is a task where the goal is to predict a **continuous, numerical output value** based on one or more input variables (features).
- In a **classification problem**, the goal is to predict a **discrete label** or **category** for a given input. Instead of calculating "how much" (like a price), you are determining "which one" (like a species or a status).

Regression problem: Example

In **regression problem** the label $y^{(i)}$ is a real number: $y^{(i)} \in \mathbb{R}$. An example is given in the dataset `concrete.csv`.

```
import pandas as pd
concrete_data = pd.read_csv("./data/concrete.csv")
n_samples = len(concrete_data)
print(f"Number of samples = {n_samples}")   # the first row (headers of data) not counted
concrete_data.head()
```

```
Number of samples = 1030
```

	cement	slag	ash	water	superplastic	coarseagg	fineagg	age	strength

	cement	slag	ash	water	superplastic	coarseagg	fineagg	age	strength
0	540.0	0.0	0.0	162.0	2.5	1040.0	676.0	28	79.99
1	540.0	0.0	0.0	162.0	2.5	1055.0	676.0	28	61.89
2	332.5	142.5	0.0	228.0	0.0	932.0	594.0	270	40.27
3	332.5	142.5	0.0	228.0	0.0	932.0	594.0	365	41.05
4	198.6	132.4	0.0	192.0	0.0	978.4	825.5	360	44.30

In this dataset, we use 8 input features are used to make prediction, that is $\mathbf{x}^{(i)} \in \mathbb{R}^8$. The label $y^{(i)}$ is a scalar (real) number, i.e. $y^{(i)} \in \mathbb{R}$.

Note Do not confuse "*regression problem*" with "*linear regression*" model. Linear regression is a mathematical model, while regression problem refers to a class of problems of predicting contiuous outputs.

Classification problem: Example

In **classification problem** the label $y^{(i)}$ is an integral number (integer type) and it takes only finite number of values. If the problem involves classification of samples into K categories, the value of $y^{(i)}$ can take only K possibilities: $y^{(i)} \in \{0, \dots K-1\}$. Apparently, it does not matter how you set the value for $y^{(i)}$ as long as there are only K possible values. However, it is conventional to start from 0 and then increase by 1. The mathematics stay the same; just formality of how you name things. An example is given in the dataset `ecg.csv`

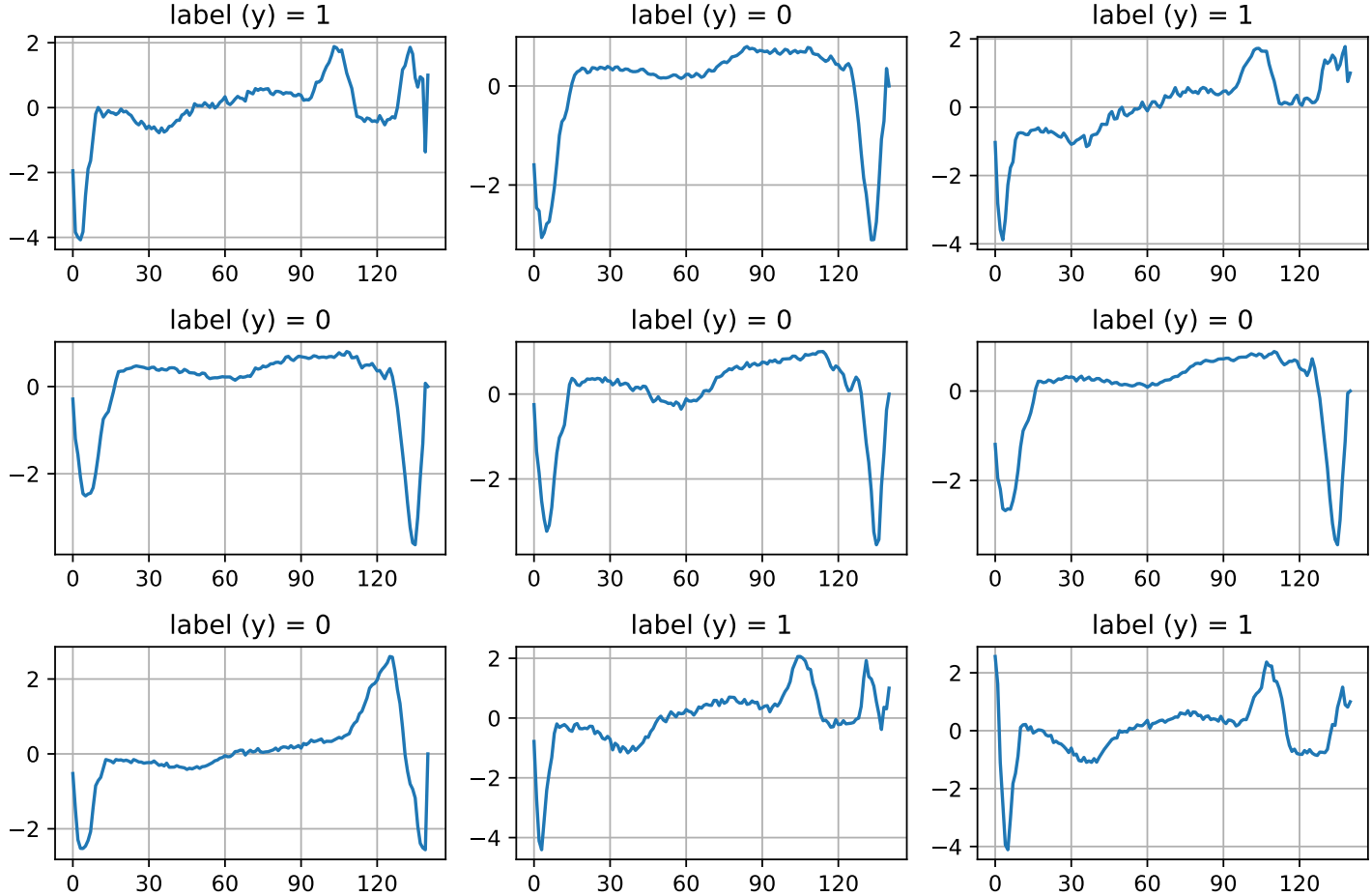
```
import numpy as np
data = np.loadtxt('./data/ecg.csv', delimiter=',')
X = data[:,:-1] # take all the rows, all the column from the first to the second last
y = data[:,-1]  # take all the rows, the last column
n_features = X.shape[1]
n_classes = len(np.unique(y))
print(f"n_features = {n_features}")
print(f"n_classes = {n_classes}")
```

```
n_features = 140
n_classes = 2
```

As this is the ECG diagram, let us draw them and show the label as on the figure title as well.

```
import matplotlib.pyplot as plt
nrows, ncols = 3, 3
plt.figure(figsize=(9, 6))
np.random.seed(seed=42)         # just for reproducibility purpose
for i in range(nrows * ncols):
    plt.subplot(nrows, ncols, i+1)
    idx = np.random.randint(low=0, high=X.shape[0])
    plt.plot(data[idx])
    plt.title(f"label (y) = {int(y[idx])}")
    plt.xticks(np.linspace(0, 120, 5))
    plt.grid(True)

plt.tight_layout()
```



3 Mathematics behind the scene

Since we want to implement these models in C++ from scratch, we need to understand the mathematics behind the scene. This is the topic of this Section. Note that you have learned the maths behind `LinearRegression` in the **Assignment 01**. We shall revisit it and proceed to logistic regression.

You will quickly recognize that although the mathematical formulation behind the logistic regression model is slightly different from that of linear regression model, the gradient descent iteration algorithm shares exactly the same formulation. For this reason, a large portion of what you have done for linear regression can easily carried over to logistic regression.

3.1 Building and training a machine learning model

You have heard a lot about the Machine Learning and all the hype about it. In my view[1], building and training a machine learning model needs 3 key components:

1. **Build or define a model**

   This step involves how we define a mapping from one space to another space. This mapping depends on a set of parameters which is called **learnable parameters** or **model parameters**.

2. **Define how we measure the model**

   To train the model or to learn the right parameters, we need a way to measure how well the model performs in relative with the given dataset. In this step, we define a loss function which measures how well the model

[1] I have taught "Intro to Machine Learning (ML) & Artificial Intelligence (AI) 4" (**ENG4200**) and "Advanced ML and AI 5" (**ENG5337**) in the last three academic years

make predictions. We define the loss function so that it serves the following intuiition. The smaller the loss function is, the better the model can make good predictions. The larger the loss function is, the worse the model can make good predictions.

3. **Train the model**

   To train a model is to look for the model parameters so that the loss function becomes smaller and smaller, leading to a minimization problem. To solve this minimization problem, we need a gradient-based method and there are many such gradient-based methods.

3.2 Gradient descent method

Imagine that the graph of a loss function

$$\mathcal{L} = \mathcal{L}(\Theta) = \mathcal{L}(\theta_1, \dots, \theta_n), \quad \Theta \in \mathbb{R}^n$$

can be visualized as a valley and the minimum occurs at the bottom of the valley. Starting from a particular position on the valley, we try to walk all the way down to the bottom. At one single position, we want to make a step so that we can actually walk down. Mathematically, one can prove that the gradient of the loss function $\nabla\mathcal{L}$ is the direction we can walk down the fastest.[2] For this reason, the gradient descent method was invented to minimize the loss function $\mathcal{L} = \mathcal{L}(\Theta)$.

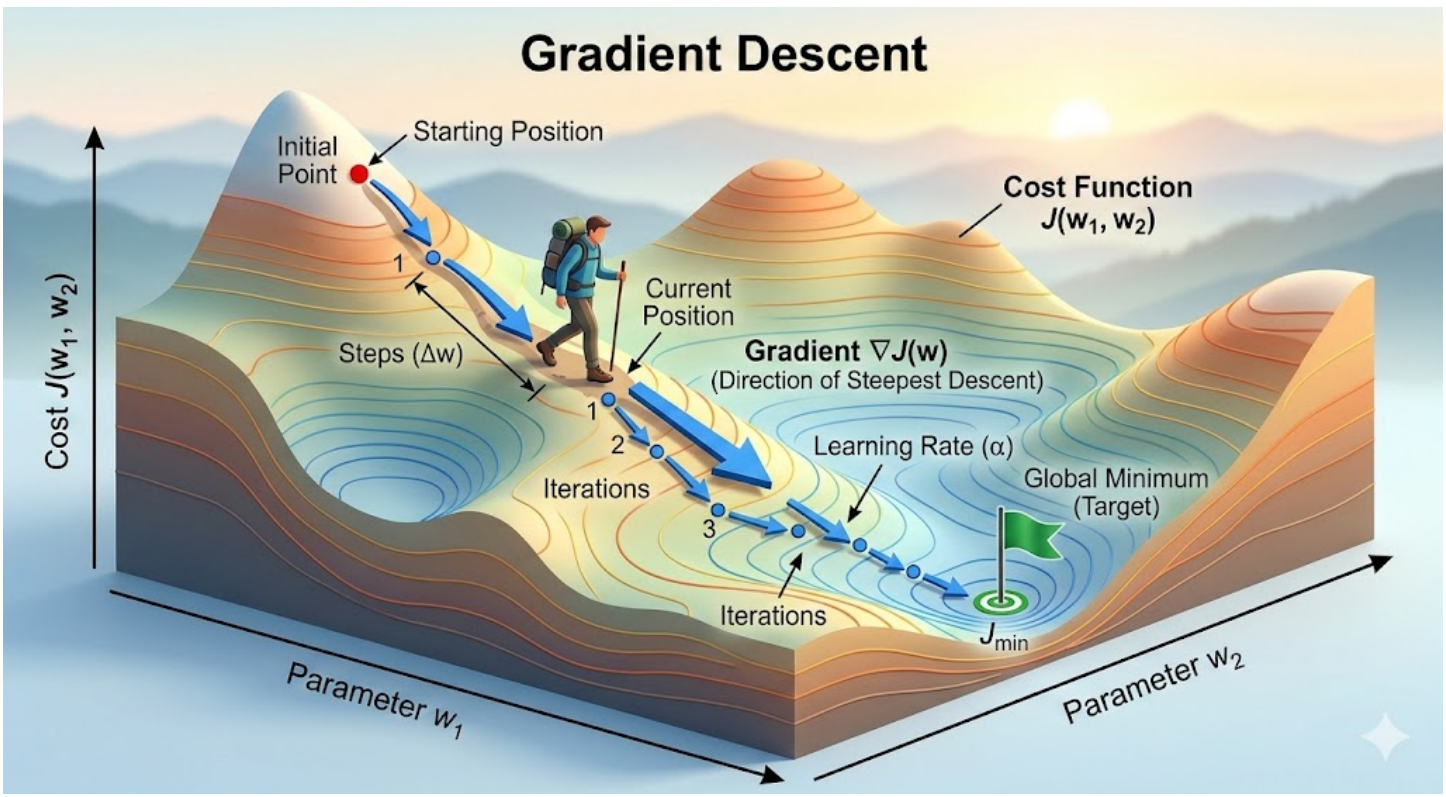


Figure 1: Illustration of the gradient descent method

Now, we want to minimize the loss function $\mathcal{L} = \mathcal{L}(\Theta)$ following the intuition presented in the above illustration figure. Assume that we start initially at the position $\Theta^{[0]}$. The aim is to compute $\Theta^{[1]}$ so that the loss function at $\Theta^{[1]}$ is smaller than the loss function evaluated at $\Theta^{[0]}$. This can be done by using the following update formula:

$$\Theta^{[1]} := \Theta^{[0]} - \alpha\nabla\mathcal{L}(\Theta^{[0]}),$$

where $\nabla\mathcal{L}(\Theta^{[0]}) \in \mathbb{R}^n$ is the gradient of the loss function $\mathcal{L}$ evaluated at $\Theta^{[0]}$ and $\alpha \in \mathbb{R}$ is the learning rate (just a scalar) dictating how far we step in the direction $\nabla\mathcal{L}(\Theta^{[0]})$.

[2] If you are interested in how this is proved, please feel free to ask me.

It turns out that the next step to walk as a human in the figure follows exactly the same strategy. Therefore, it leads to the iteration process

$$\Theta^{[t+1]} := \Theta^{[t]} - \alpha \nabla \mathcal{L}(\Theta^{[t]}), \quad \forall t = 0, 1, 2, ... \tag{2}$$

Equation 2 is written in the vector form and can be rewritten in the unpacked form as follows

$$\begin{aligned}
\theta_1^{[t+1]} &:= \theta_1^{[t]} - \alpha \frac{\partial \mathcal{L}}{\partial \theta_1}(\theta_1^{[t]}, ..., \theta_2^{[t]}), \\
\theta_2^{[t+1]} &:= \theta_2^{[t]} - \alpha \frac{\partial \mathcal{L}}{\partial \theta_1}(\theta_1^{[t]}, ..., \theta_2^{[t]}), \\
&\text{etc.} \\
\theta_n^{[t+1]} &:= \theta_n^{[t]} - \alpha \frac{\partial \mathcal{L}}{\partial \theta_1}(\theta_1^{[t]}, ..., \theta_n^{[t]}).
\end{aligned}$$

This is the key algorithm we need to implement for our project assignment.

3.3 Linear Regression with multiple features

In this section, we study:

- Description of a regresison problem
- Linear regression model
- Loss function for training the linear regression model
- Gradient of loss function

Problem description

We are given a dataset $\mathcal{D}$ with m samples and each sample has n input features $\mathbf{x}^{(i)} = (x_1^{(i)}, ..., x_n^{(i)}) \in \mathbb{R}^n$ and one output label $y \in \mathbb{R}^n$ – see Equation 1 again

$$\mathcal{D} = \left\{ (\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), ..., (\mathbf{x}^{(m)}, y^{(m)}) \right\} \tag{3}$$

This dataset can be written in two matrices

$$\mathbf{X} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & ... & x_n^{(1)} \\ x_1^{(2)} & x_2^{(2)} & ... & x_n^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(m)} & x_2^{(m)} & ... & x_n^{(m)} \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix} \tag{4}$$

Our task is to build a model, which is actually just a function or a mapping, to predict a new output $\hat{y}$ given a new sample (data point) $\mathbf{x}$. That is we need to construct the function $f = f(\mathbf{x})$ so that it fits the data very well. If the model can be visualized, we expect the graph $y = f(\mathbf{x})$ should fit the data points in the high-dimensional space $\mathbf{R}^{n+1}$ very well. Now, we go through three steps mentioned in Section 3.1.

Linear regression model

The linear regression model is defined by a simple linear relationship between input $\mathbf{x}$ and output y:

$$\hat{y} = f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b.$$

The parameters $\mathbf{w} = (w_1, ..., w_n)$ are called **weights** and the parameter b is called **bias**. This linear regression model is fully determined by the **learnable parameters** $\mathbf{w} = (w_1, ..., w_n)$ and b. **Our objective** is to seek these *learnable parameters* $w_1, ..., w_n$ and b.

Loss function

To measure how well the model is, we use the **Mean Square Error** (MSE) loss function:

$$\begin{aligned}\mathcal{L}(\mathbf{w}, b) &= \frac{1}{m}\sum_{i=1}^{m}(\hat{y}(\mathbf{x}^{(i)}) - y^{(i)})^2 = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)}\right)^2 \\ &= \frac{1}{m}\sum_{i=1}^{m}\left(\mathbf{w}\cdot\mathbf{x}^{(i)} + b - y^{(i)}\right)^2\end{aligned} \tag{5}$$

Note that we don't use $2m$ in the denominator as in the Assignment 01. This is just to prove that the minimium point is exactly the same, only the minimum value is scaled by a factor $1/2$. For this reason, the gradient $\nabla\mathcal{L}$ will be also scaled (see below).

It is immediately clear that the loss function $\mathcal{L}$ defined as above is just a function of $\mathbf{w}$ and b because the values $\mathbf{x}^{(i)}$ and $y^{(i)}$ for all $i = 1, \dots, m$ are given in the dataset $\mathcal{D}$ (e.g., in the `.csv` file).

We recall here "training the model" means "looking for $\mathbf{w}$ and b" and the way to do this is to implement the gradient descent method given by the iteration formula Equation 2. In the current case, the variable Θ corresponds to the model parameters $\mathbf{w}$ and b. To implement the update rule given by Equation 2, we need to compute the gradient $\nabla\mathcal{L}$ which refers to derivatives of $\mathcal{L}$ with respect to $w_1, w_2, \dots, w_n$ and to b:

$$\nabla\mathcal{L} = \left(\frac{\partial\mathcal{L}}{\partial w_1}, \cdots, \frac{\partial\mathcal{L}}{\partial w_n}, \frac{\partial\mathcal{L}}{\partial b}\right)$$

Therefore, we need to compute the gradient of the loss function $\nabla\mathcal{L}$ to perform the gradient descent iteration algorithm.

Gradient of loss function

The gradient of $\mathcal{L}$ given by Equation 5 is computed as follows[3] – If you are interested in the derivation, please refer `derivation.html`.

$$\frac{\partial\mathcal{L}}{\partial w_1} = \frac{2}{m}\sum_{i=1}^{m}(\hat{y}(\mathbf{x}^{(i)}) - y^{(i)})x_1^{(i)}$$

so on

$$\frac{\partial\mathcal{L}}{\partial w_n} = \frac{2}{m}\sum_{i=1}^{m}(\hat{y}(\mathbf{x}^{(i)}) - y^{(i)})x_n^{(i)}$$

$$\frac{\partial\mathcal{L}}{\partial b} = \frac{2}{m}\sum_{i=1}^{m}(\hat{y}(\mathbf{x}^{(i)}) - y^{(i)})\cdot 1$$

Note that since we have m instead of $2m$ in the denominator of Equation 5, the gradient is also scaled, and thus we see number 2 in the nominator of $\frac{2}{m}$ as compared to $\frac{1}{m}$ in the Assignment 01. By plugging these equations in Equation 2, we obtain the following update rule

$$w_1^{[t+1]} := w_1^{[t]} - \frac{2}{m}\sum_{i=1}^{m}\left[(\mathbf{w}^{[t]}\cdot\mathbf{x}^{(i)} + b^{[t]}) - y^{(i)}\right]x_1^{(i)}$$

so on

$$w_n^{[t+1]} := w_n^{[t]} - \frac{2}{m}\sum_{i=1}^{m}\left[(\mathbf{w}^{[t]}\cdot\mathbf{x}^{(i)} + b^{[t]}) - y^{(i)}\right]x_n^{(i)}$$

$$b^{[t+1]} := b^{[t]} - \frac{2}{m}\sum_{i=1}^{m}\left[(\mathbf{w}^{[t]}\cdot\mathbf{x}^{(i)} + b^{[t]}) - y^{(i)}\right]\cdot 1$$

[3] You can try to carry out the derivation. It is surprisingly easy to be honest. Clever students like you will definitely do this

3.4 Logistic Regression with multiple features

In this section, we study:

- Description of a classification problem
- Logistic regression model
- Loss function for training the logistic regression model
- Gradient of loss function

Problem description

The problem description for a classification problem is quite similar to the description of a regression problem in terms of the given dataset and the ultimate goal prediction, except that the label y is not continous value. For a K-category classification problem, the label y can take one of the K finite values $\{0, \dots, K-1\}$. Examples are:

- For binary classification problem of ECG data, we have $K = 1$ and the label can take value of either 0 or 1.
- For hand-written digit classification, we have 10-category classification with $K = 9$ and the label can take one value of the 10 finite values $\{0, \dots, 9\}$

For this reason, the dataset notation Equation 3 and the matrix-presentation of this dataset Equation 4 should be the same except that the label $y^{(i)}$ for all the samples can take only one value in the finite set $\{0, \dots, K-1\}$ for the K-category classification problem.

4 Task

You need to design a header-only library that is implemented in an Object-oriented Programming fashion so that it mimics the scikit-learn library in Python. Particularly, the following are the must-do:

4.1 Basic tasks

1. Decide your own OOP design to implement the library and program in general. The design involves how you want to organize classes, inheritence, choices of data types and data storage. **Note** that a functioning program is not necessary a well-designed programe. You need to make your own decision! Try to function as a software engineer !!!
2. Build two classes `LinearRegression` and `LogisticRegression` that mimic their counterparts in Python. These classes must have two important member functions `fit()` and `predict()` with appropriate function parameters.
3. Decide on how you want to read the `.csv` file. Using either functions or object-oriented programming is fine. Make sure you give convincing reasons for your choice in the report.
4. Add these two classes into the namespace `linear_model`. This `linear_model` is again a namespace inside a bigger namespace `sklearn_cpp`. This step is to mimic the important statement in Python. As you can see, the `linear_model` is a submodule of the module `sklearn`.

   ```
   from sklearn.linear_model import LinearRegression
   from sklearn.linear_model import LogisticRegression
   ```

5. Write a main program to apply these two classes to solve two problems:

   - Predict concrete strength using linear regression – just like example in Python
   - Predict whether a patient is fine or not given an ECG data – just like example in Python

4.2 Build the header-only library

1. Based on the code you have achieved in Section 4.1, build [header-only] library by following my provided example. This is a fantastic [header-only] library that you can look at: xtensor. All the header files can be found at header file source code.
2. Write a main program that use the [**header-only**] library you have just built to test whether the library works.

5 Marking scheme

- Basic tasks: 70/100
  - Good OOP program design: 15/100
  - Build two classes and test them correctly: 40/100
  - Main program to apply the two built classes to two applications: 15/100
- Build the header-only library: 10/100
- Report: 0/10